

Parte 4.1 - Investigación

¿Qué es una IDP (Internal Developer Platform)?

Una IDP (Internal Developer Platform) es una plataforma interna diseñada para que los desarrolladores de una organización puedan construir, probar, desplegar y operar sus aplicaciones de forma más eficiente, uniforme y autoservicio. El objetivo principal de una IDP es abstraer la complejidad infraestructural y operativa, brindando a los equipos de desarrollo herramientas y procesos unificados.

Usos de una IDP:

- Estandarización: Los desarrolladores trabajan bajo un mismo entorno estandarizado, lo que facilita la colaboración y acelera la curva de aprendizaje.
- Escalabilidad: Permite a los equipos mantener la coherencia en el desarrollo, pruebas y despliegue en un entorno con múltiples microservicios.
- Autoservicio: Los desarrolladores pueden aprovisionar entornos, recursos y configuraciones sin depender del equipo de infra, gracias a una capa abstracta de infraestructura ya configurada.
- Reducción de costo y tiempo: Al automatizar procesos repetitivos, se reducen tiempos de espera y costos operativos.
- Mayor confiabilidad: Establece prácticas recomendadas y herramientas preconfiguradas, mejorando la calidad y robustez del software.

Creación de la imagen Docker para Visual Studio Code Server

El objetivo es crear un contenedor con VS Code Server accesible vía navegador web, que sea liviano, agnóstico al SO host y consuma pocos recursos.

Requerimientos:

- Tener instalado `code-server` en la imagen.
- Definir el puerto por el que se accederá a la aplicación web (por defecto `8080`).
- Configurar un usuario/contraseña o un mecanismo de autenticación seguro.
- Persistir la configuración del entorno y el código fuente mediante volúmenes.

¿Qué imagen usar de base?

Podemos partir de una imagen ligera de Linux, como `ubuntu:latest` o `alpine`, aunque para simplificar es común usar la imagen oficial de `code-server` proporcionada por Coder (<https://hub.docker.com/r/codercom/code-server>). Esta ya viene optimizada y con todas las dependencias. Por ejemplo: `FROM codercom/code-server:latest`

¿Qué puertos se deben exponer?

Por defecto, `code-server` escucha en el puerto `8080`. Por lo tanto, expondremos el puerto `8080`.

¿Archivo de configuración?

`code-server` puede configurarse a través de un archivo `config.yaml` o variables de entorno. Este archivo se suele ubicar en `~/ .config/code-server/config.yaml`. Por ejemplo:

```
bind-addr: 0.0.0.0:8080
auth: password
password: "mi_password_segura"
cert: false
```

Este archivo puede copiarse en la imagen o bien montarse desde el host.

Dockerfile (ejemplo):

```
FROM codercom/code-server:latest

# Opcionalmente, copiar un archivo de configuración personalizado
COPY config.yaml /home/coder/.config/code-server/config.yaml

# Ajustar permisos si es necesario
RUN chown -R coder:coder /home/coder/.config

# Exponer el puerto
EXPOSE 8080

# El CMD por defecto es el de la imagen oficial, que inicia code-server
```

Docker Compose para ejecutar code-server Un `docker-compose.yaml` podría tener la siguiente estructura:

```
version: '3.8'
services:
  code-server:
    image: my-code-server:latest
    container_name: code-server
    restart: unless-stopped
    ports:
      - "8080:8080"
    volumes:
      # Volumen para persistir extensiones, configuraciones y proyectos
      - ./projects:/home/coder/projects
      - ./config.yaml:/home/coder/.config/code-server/config.yaml
    environment:
      # Podemos setear la password vía entorno si no se desea en YAML
      # CODE_SERVER_PASSWORD: "mi_password_segura"
      # command: ["code-server", "--bind-addr", "0.0.0.0:8080",
      # "/home/coder/projects"]
```

En este ejemplo:

`ports` mapea el puerto interno `8080` al `8080` del host. `volumes` asegura que el directorio de proyectos y el archivo de configuración persistan en el host. `environment` o `config.yaml` definen credenciales, opciones de seguridad y comportamiento del server.

Parte 4.2 - Parte Práctica

Levantar una base de datos SQL Server con Docker

Comando Docker: Para levantar una instancia de SQL Server en Linux (usando la imagen oficial de Microsoft):

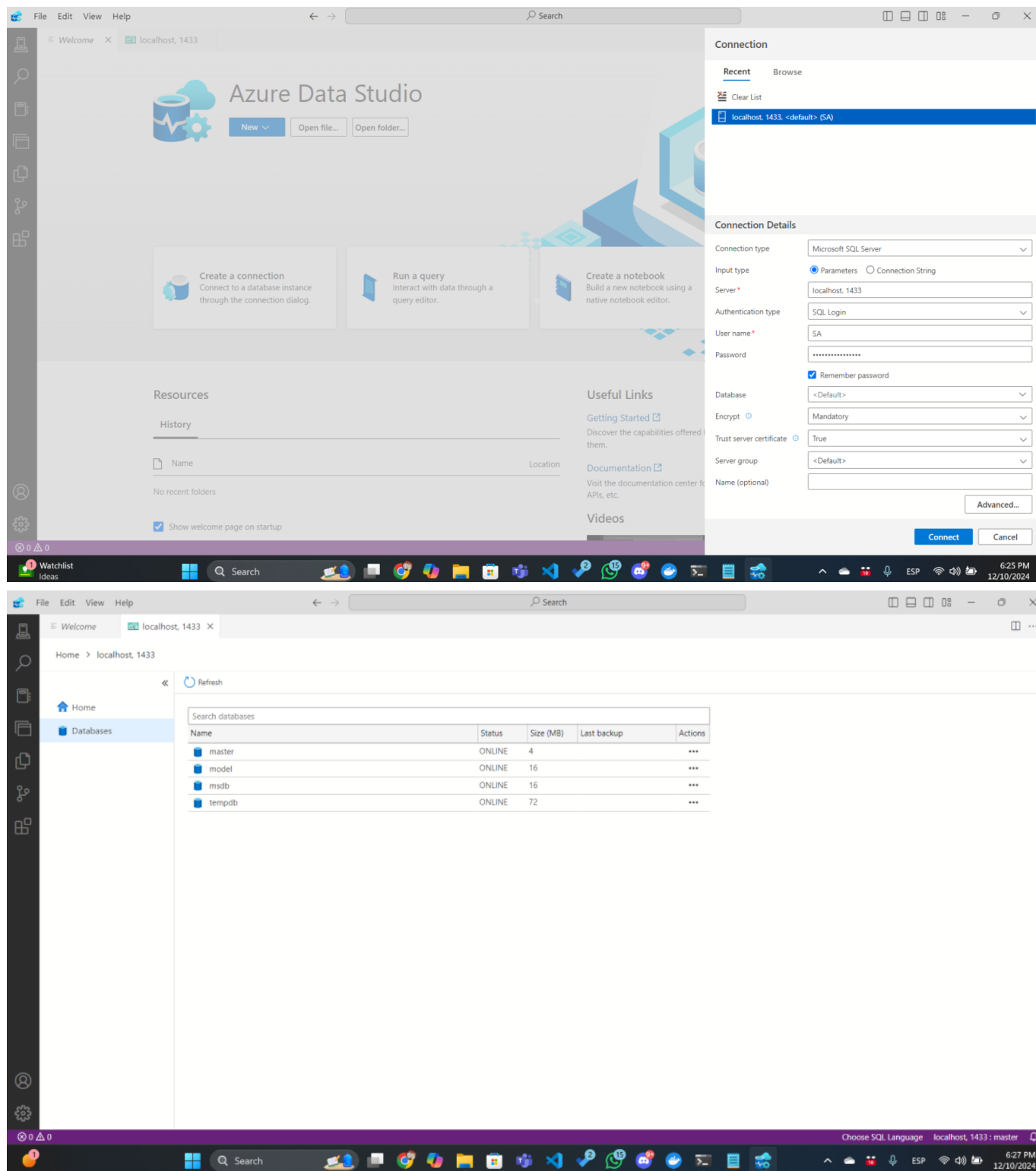
```
docker run -e "ACCEPT_EULA=Y" -e "SA_PASSWORD=Your_password123" -p 1433:1433 --name sqlserver -d mcr.microsoft.com/mssql/server:latest
```

Explicación:

- `ACCEPT_EULA=Y`: Acepta los términos de la licencia.
- `SA_PASSWORD=Your_password123`: Establece la contraseña del usuario SA.
- `-p 1433:1433`: Expone el puerto por defecto de SQL Server.
- `mcr.microsoft.com/mssql/server:latest`: Imagen oficial de SQL Server.

Conexión con DBeaver o Azure Data Studio

1. Descargar e instalar Azure Data Studio.
2. Crear una nueva conexión.
3. Host: `localhost` o la IP del servidor donde corre el contenedor.
4. Puerto: `1433`.
5. Usuario: `SA`.
6. Contraseña: `Your_password123`.
7. Conectar y, si es necesario, aceptar certificados o configuraciones.



Usar Docker Compose para la base de datos

Podemos definir un `docker-compose.yml` para levantar la base de datos con un solo comando `docker-compose up -d`:

```
version: '3.8'
services:
  sqlserver:
    image: mcr.microsoft.com/mssql/server:latest
    container_name: SOSQL
    environment:
```

```
ACCEPT_EULA: "Y"
SA_PASSWORD: "Your_password123"
ports:
  - "1434:1434"
volumes:
  - ./data:/var/opt/mssql/data
restart: unless-stop
```

